

Vaultic Research Roadmap

Oct 7, 2026 · @Aziz Ahmad

Overview

Vaultic grows from the FYP-1 fraud detection app into a research framework whose main claim is a **missing-view-aware adaptive fusion** of five kinds of fraud evidence, evaluated with no temporal leakage.

Item	Name
System name	Vaultic
Research / thesis title	Vaultic: An Adaptive Multi-View Framework for Explainable, Uncertainty-Aware and Drift-Aware Financial Fraud Detection
Main method name	Missing-View-Aware Adaptive Fusion (MVAF)
Primary dataset	IEEE-CIS Fraud Detection (Vesta), training file only

The five evidence views

1. **Tabular**: what the transaction itself looks like (XGBoost/LightGBM).
2. **Behavioral**: how unusual it is for this customer (deviation, velocity, novelty features).
3. **Temporal**: what happened just before it (sequence model over the customer's last transactions).
4. **Relational**: who and what it is connected to (dynamic entity graph, graph features, GNN).
5. **Anomaly**: how far it sits from normal behavior, without labels (Isolation Forest, autoencoder).

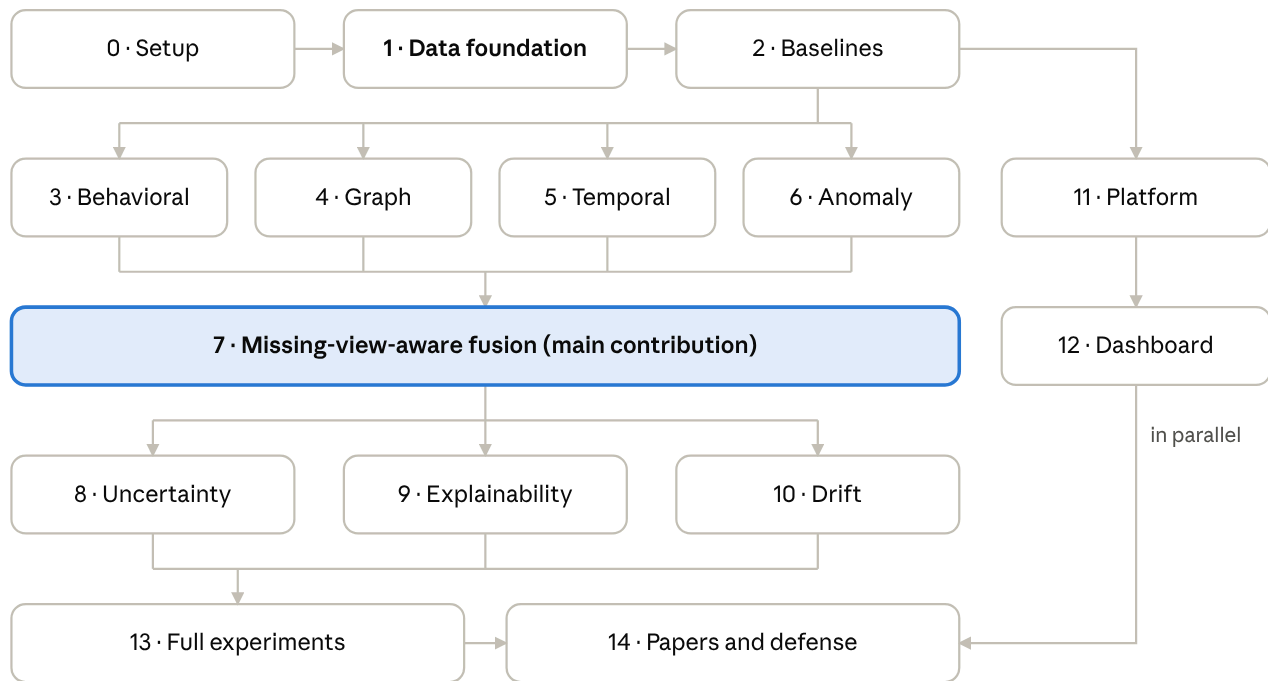
The five research pillars

#	Pillar	Core question	Paper
1	Missing-view-aware fusion	Does fusion that knows which evidence is missing beat stacking when evidence is incomplete?	Paper 2 (main)
2	Leakage-free dynamic graphs	How much of reported GNN fraud performance comes from temporal leakage?	Paper 1
3	Drift under delayed labels	Can label-free signals detect drift before delayed fraud labels reveal it?	Paper 3
4	Budget-aware uncertainty routing	With a fixed analyst budget, does uncertainty-aware routing catch more fraud value than ranking by risk?	Paper 2
5	Faithful multi-view explanations	Are explanations of a fused model faithful, or only convincing?	Paper 2

How to use this roadmap

- Each phase lists its **goal**, **tasks** (tick them off), **deliverables** and an **exit gate**.
- Do not start a research phase until the gate before it passes. Phases 11–12 (platform and dashboard) run in parallel with the research phases.
- Every number that goes in a paper must come from the evaluation harness built in Phase 1, never from a notebook run by hand.

Every research phase builds on one data foundation; the platform runs alongside



Paper 1 comes from phase 4, Paper 3 from phase 10, Paper 2 from phases 7–9.

Phase map · 15 phases and their dependencies

Phase 1 gates everything: if the customer ID or the leakage rules are wrong, every later result is wrong too.

Extension phases added later: 15 (LLM copilot) builds on 9 and 11; 16 (adversarial robustness) and 17 (federated training) build on 7. Phase 15 is core; 16 and 17 are stretch goals.

Research questions and hypotheses

The project answers six research questions; each has one testable hypothesis, one experiment and one primary metric.

RQ	Question	Hypothesis	Experiment	Primary metric
RQ1	Do behavioral, temporal, relational and anomaly views improve on transaction-level features alone?	H1: each added view raises PR-AUC over the strong tabular baseline, with non-overlapping 95% CIs for at least two views	Ablation E3-E9	PR-AUC
RQ2	Does missing-view-aware fusion beat static fusion and stacking?	H2: MVAF beats LR and LightGBM stacking overall, and by a larger margin on transactions with missing views	E10-E11	PR-AUC, recall@1% FPR
RQ3	How much does temporal leakage inflate graph-based fraud detection results?	H3: a static full graph reports a significantly higher PR-AUC than a strictly temporal graph	E6 (Paper 1)	PR-AUC gap
RQ4	Can label-free signals detect drift earlier than label-based detectors when labels are delayed?	H4: with a 30-day label delay, PSI/disagreement detectors flag injected drift earlier than ADWIN on errors	E13 (Paper 3)	detection delay (days), false alarms
RQ5	Does uncertainty-aware routing catch more fraud value under a fixed analyst budget?	H5: at equal budget K, routing by expected loss plus uncertainty catches more fraud value than top-K risk	E12	fraud value caught per K reviews
RQ6	Are the framework's explanations faithful and stable?	H6: removing the top-k SHAP features lowers risk significantly faster than removing random features	E15	deletion-curve AUC

Extension questions (Phases 15–17)

RQ	Question	Hypothesis	Experiment	Primary metric
RQ7	Does a continuous-time dynamic GNN beat snapshot GraphSAGE once leakage is removed?	H7: under the strict temporal setting, the memory-based model gains PR-AUC on uids with dense entity links	E18	PR-AUC
RQ8	Can an LLM copilot answer analyst questions using only system evidence?	H8: tool-grounded answers have far fewer unsupported claims than an ungrounded LLM given the same case	E20	unsupported-claim rate, answer accuracy
RQ9	How robust is the framework to fraudsters who adapt to it?	H9: multi-view fusion loses less recall than the tabular model under realistic evasion, and adversarial retraining recovers most of the loss	E21	recall under attack
RQ10	Can institutions that cannot share data jointly train the fusion model?	H10: federated training reaches most of the centralized PR-AUC and clearly beats each bank training alone	E22	PR-AUC vs centralized

If a hypothesis fails, report it. A negative result with a clean method is still publishable and defensible.

Dataset realities and constraints

Use only the IEEE-CIS **training** files, split by time; the Kaggle test labels are not public, so the test files cannot be used for evaluation.

Figures below are approximate from general knowledge of the dataset; confirm each one in Phase 1 and record the real values in the data report.

Fact	Approximate value	What it means for the project
Training transactions	~590,000	Large enough for GNNs and sequence models, but graph work needs sampling
Fraud rate	~3.5%	Imbalanced: PR-AUC is the primary metric, never accuracy
Time span	~6 months (TransactionDT in seconds from an unknown start)	Enough for monthly drift analysis; no real calendar dates or local hours
Rows with identity/device data	~24%	The relational and device views are missing for most rows (Pillar 1)
Customer ID	Not provided	Must be reconstructed (Phase 1)
Feature meaning	Masked (C1-C14, D1-D15, M1-M9, V1-V339, id_01-id_38)	Reason codes must describe masked features honestly
Merchant / location	Only ProductCD (5 values) and masked addr1/addr2	No merchant graph, no travel-speed features

Secondary datasets (limited roles only)

Dataset	Use it for	Do not claim
PaySim (synthetic mobile money)	Sanity checks for drift injection and velocity features	That results generalize to real banks
Elliptic (Bitcoin transaction graph)	Checking that the temporal-graph leakage finding (Paper 1) also appears on a public graph benchmark	Cross-domain generalization of the full framework

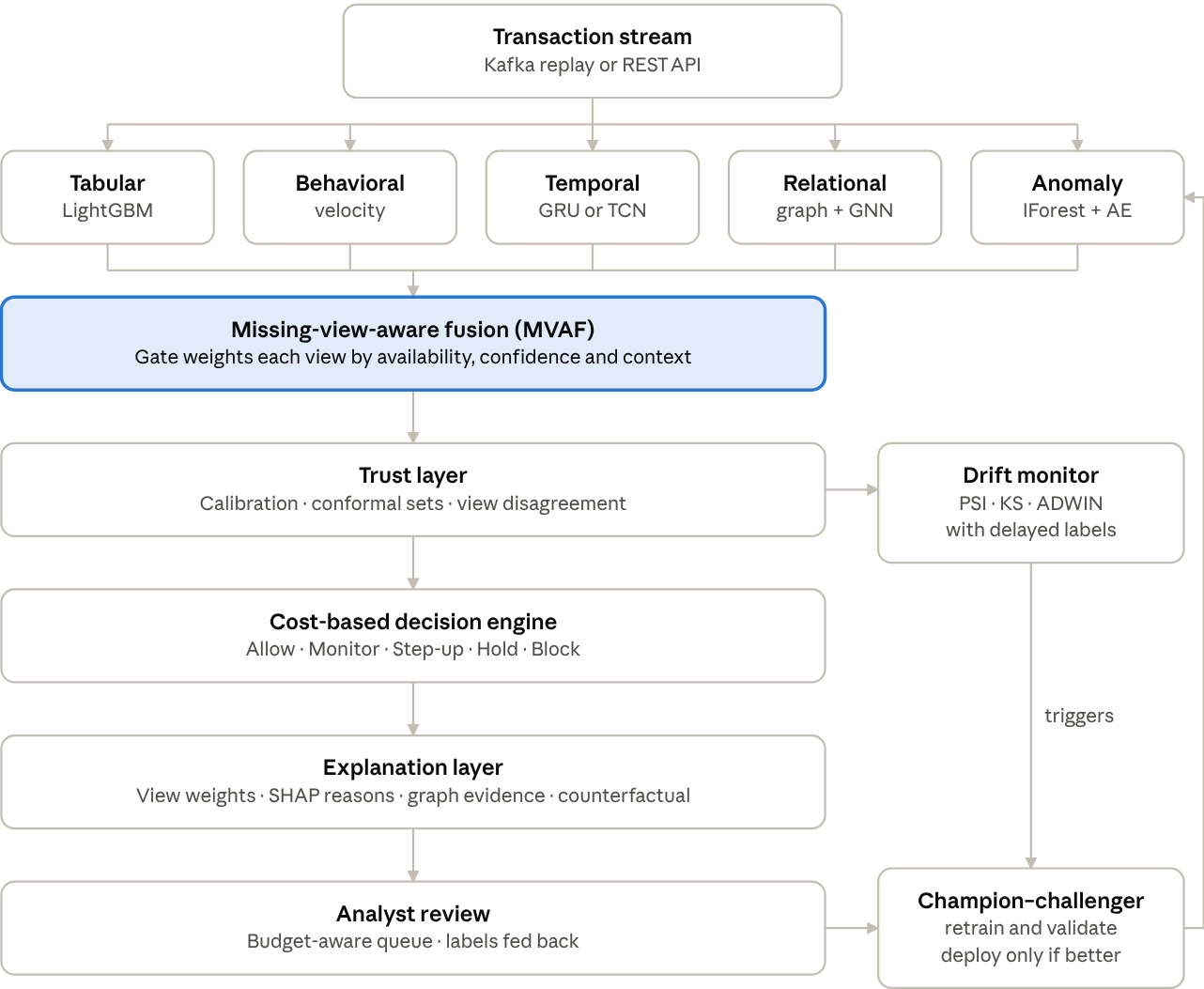
Data rules

- Never edit the raw CSVs; keep them read-only under `data/raw/` and version derived files with DVC.
- Sort everything by TransactionDT before any feature is built.
- In papers, describe the data as "anonymized e-commerce transactions released by Vesta for the IEEE-CIS competition", never as private bank data.

System architecture and tech stack

Every transaction flows through five evidence views into one fusion model, then a trust layer, a cost-based decision and an explanation, and analyst feedback flows back into controlled retraining.

Five evidence views feed one fusion model, and feedback closes the loop



Vaultic architecture · 5 views, fusion, trust, decisions, feedback loop

The drift monitor and analyst labels both feed the champion-challenger step, and only a validated model returns to the views.

Final tech stack (all free)

Layer	Tool	Why this one
Language	Python 3.11	Already used in Vaultic
Tabular models	XGBoost, LightGBM, scikit-learn	Strong, fast baselines

Layer	Tool	Why this one
Deep learning	PyTorch	GRU/TCN/Transformer, autoencoder, fusion network
Graph learning	PyTorch Geometric (HeteroData, NeighborLoader)	Heterogeneous GraphSAGE with sampling
Graph analysis	NetworkX (python-igraph if slow)	Graph features, components, communities
Explainability	SHAP, DiCE	Feature attributions, counterfactuals
Uncertainty	MAPIE or hand-written conformal code	Conformal prediction sets
Drift	river (ADWIN, Page-Hinkley), scipy (KS)	Streaming and batch drift tests
Experiment tracking	MLflow, DVC	Reproducible runs and data versions
Streaming	Apache Kafka (single-node KRaft in Docker)	Real-time event pipeline
Online features	Redis	Rolling counters per customer and entity
API	FastAPI	Scoring, explanation and feedback endpoints
Database	PostgreSQL	Transactions, scores, cases, feedback, model registry
Frontend	React + Vite, Recharts, Cytoscape.js, TanStack Table	Dashboard, charts, fraud-network graph
Load testing	Locust	Throughput and p50/p95/p99 latency
Packaging and CI	Docker Compose, GitHub Actions, pytest	One-command startup, automatic tests

Extension tools, also free: TabPFN and FT-Transformer (modern baselines), PyTorch Geometric temporal modules (TGN-style model), Ollama with an open-weight model (LLM copilot), Flower (federated training).

On Windows, run Kafka, Redis and PostgreSQL through Docker Desktop with WSL2 rather than installing them natively.

Repository structure and experiment logging

One repository holds the research code, the platform and the papers, so every figure in a paper can be traced to the exact code and config that produced it.

```

vaultic/
  data/
    raw/           # IEEE-CIS CSVs, read-only, tracked by DVC
    interim/       # merged, sorted, uid added
    features/      # point-in-time feature tables (parquet)
  src/vaultic/
    data/          # loading, merging, uid reconstruction, splits
    features/      # tabular, behavioral, velocity, graph features
    views/         # tabular, temporal, graph, anomaly models
    fusion/        # MVAf gating network + fusion baselines
    trust/         # calibration, conformal, disagreement, routing
    explain/       # SHAP, counterfactuals, faithfulness, reason codes
    drift/         # PSI, KS, ADWIN, label-delay simulator
    learning/      # feedback, active learning, champion-challenger
    eval/          # harness, metrics, bootstrap, significance tests
  experiments/
    configs/       # one YAML per experiment (EXP-001.yaml ...)
    runs/          # outputs per run (git-ignored, logged to MLflow)
  platform/
    api/           # FastAPI service
    stream/        # Kafka producer, consumers, scorer
    online_features/ # Redis feature updates
    db/            # PostgreSQL schema + migrations
  dashboard/      # React app
  tests/          # unit, leakage and parity tests
  research/
    literature/    # paper notes + review spreadsheet
    tables/ figures/ # generated by scripts, never by hand

```

```
papers/paper1/ paper2/ paper3/  thesis/
docker-compose.yml
dvc.yaml
README.md
```

Experiment logging standard

Every experiment is one YAML config and one command: `python -m vaultic.eval.run experiments/configs/EXP-023.yaml`.

Field	Example
id	EXP-023
question	RQ1: does adding graph features help?
views and features	tabular + behavioral + graph
split	train days 0-120, gap 7, val 128-150, test 151-182
label delay (days)	30
seeds	0, 1, 2, 3, 4
code version	git commit hash (logged automatically)
data version	DVC hash (logged automatically)

Each run saves `config.yaml`, `metrics.json` (mean, std and 95% CI per metric), `predictions.parquet` (id, time, label, score per view, final score) and its plots, all logged to MLflow. Keep a one-line entry per experiment in `research/experiment_log.md`: id, date, result, and what you concluded.

Phase 0: Setup, migration and literature review

Goal: a safe, versioned starting point, the FYP-1 app preserved, and a literature review that confirms the research gaps before any modelling starts.

Tasks: safe start

- ☐ Copy the whole `E:\old work\fyp 11` folder to a second drive before any more changes.

- ☐ Finish the database rebuild, check it, then tag the current code `v1-fyp1` so the FYP-1 version can always be shown.
- ☐ Fix the missing `alert_detail.html` template so FYP-1 runs fully.
- ☐ Create the new `vaultic/` repository layout from the structure above; push to a private GitHub repo shared with Hamza and Roshan.
- ☐ Set up Python 3.11 venv, `requirements.txt` with pinned versions, pre-commit (black, ruff), pytest.
- ☐ Install Docker Desktop with WSL2; confirm `docker compose up` runs a test PostgreSQL container.
- ☐ Set up MLflow (local file store) and DVC; add the raw CSVs to DVC.
- ☐ Write `CONTRIBUTING.md` : branch per feature, pull request + one review before merging to `main`.

Tasks: literature review

- ☐ Collect 30–40 papers in Zotero across six topics: fraud detection on IEEE-CIS, graph fraud detection, temporal/sequence fraud models, ensemble and fusion with missing modalities, concept drift with delayed labels, conformal prediction and XAI evaluation.
- ☐ Fill one spreadsheet row per paper: title, year, venue, dataset, split type (random or temporal), models, metrics, leakage risk noticed, limitation, relevance to which pillar.
- ☐ For every graph paper, record **how the graph was built and split**; this column is the evidence base for Paper 1.
- ☐ Write a 2-page gap summary per pillar: what exists, what is missing, how Vaultic addresses it. Rewrite the pillar questions if the literature shows one is already solved.
- ☐ Meet the supervisor with the gap summary and the RQ table; get written approval of the scope.

Deliverables

- Repo with CI running, tag `v1-fyp1` , literature spreadsheet, gap summary, supervisor approval.

Exit gate

- A teammate can clone the repo and run the tests in under 15 minutes.
- Each pillar has at least five related papers and a written gap statement.

Phase 1: Data foundation

Goal: a reconstructed customer ID, a time-based split, leakage-free features and one evaluation harness that every later phase uses. Everything else depends on this phase being right.

1.1 Load and order

- ☐ Load `train_transaction.csv` and `train_identity.csv`; left-join on TransactionID.
- ☐ Add `has_identity` (1 if the identity row exists) and `day = floor(TransactionDT / 86400)`.
- ☐ Sort by TransactionDT; save to `data/interim/merged.parquet` with reduced dtypes (float32, category).
- ☐ Write a data report: row counts, fraud rate per month, missing rate per column group, `has_identity` rate per month.

1.2 Customer ID reconstruction

- ☐ Build `uid = card1 + addr1 + (day - D1)`. D1 behaves like "days since the card was first used", so `day - D1` is roughly constant for one card holder.
- ☐ Build variants: `uid2 = uid + P_emaildomain`, `uid_card = card1 + card2 + card3 + card5` (fallback).
- ☐ Validate each variant: number of IDs, transactions per ID, share of IDs whose transactions are all fraud or all legit (higher purity = better ID), fraud rate for IDs seen once.
- ☐ Pick one variant as `uid` and write down why, with numbers. This choice becomes a stated limitation in every paper.

1.3 Time-based split and label delay

- ☐ Fixed split (adjust to the real day range): train days 0–120, 7-day gap, validation days 128–150, test days 151–182.
- ☐ Rolling split for drift work: train on months 1–3, test month 4; then 1–4 → 5; then 1–5 → 6.
- ☐ Label delay rule: a past transaction's fraud label may be used at time t only if $its_time + L \leq t$ (default $L = 30$ days). Any feature built from labels (neighbor fraud rate, customer fraud history) must obey it.

1.4 Point-in-time feature pipeline

- ☐ Every feature for a transaction at time t is computed only from transactions strictly before t (pandas `groupby(...).rolling(..., closed='left')` or an explicit loop).
- ☐ Frequency encodings and target encodings are fitted on the training period only and applied forward.
- ☐ Leakage test 1: delete all rows after time t , rebuild features, confirm features at or before t are identical.
- ☐ Leakage test 2: shuffle labels of future rows, confirm no feature at time t changes.
- ☐ Put both tests in `tests/test_leakage.py` and run them in CI on a 50,000-row sample.

1.5 Evaluation harness

- ☐ `run(config)` → trains, predicts, saves outputs, logs to MLflow (format in the logging standard above).
- ☐ Metrics: PR-AUC, ROC-AUC, recall at 1% and 5% FPR, precision@K, F1 at the validation-chosen threshold, Brier score, ECE, cost (Phase 13 cost model).
- ☐ Five seeds per run; 1,000-sample bootstrap 95% CI on the test set.
- ☐ Paired comparison helper: paired bootstrap for PR-AUC difference between two runs.
- ☐ The decision threshold is always chosen on validation, never on test.

Deliverables

- `merged.parquet`, uid report, split definition file, feature pipeline, harness, leakage tests passing.

Exit gate

- Same config + same seed gives identical metrics twice.
- Both leakage tests pass in CI.
- The uid choice is documented with numbers.

Phase 2: Baselines

Goal: a ladder of baselines ending in a strong, well-tuned tabular model, so every later gain is measured against something hard to beat.

ID	Baseline	Features	Purpose
B1	Logistic regression	Raw numeric + one-hot ProductCD, scaled	Simplest reference
B2	Random forest	Raw	Classic ML reference
B3	XGBoost (raw)	Raw columns, no engineering	What most student projects report
B4	LightGBM (raw)	Raw	Second boosting reference
B5	Strong tabular	Raw + frequency encodings + point-in-time uid aggregations + V-column reduction	The baseline reviewers expect; this is the "tabular view" used later
B6	FYP-1 Vaultic models	XGBoost + per-user Isolation Forest as built in FYP-1	Shows progress from FYP-1

Modern baselines (added): reviewers increasingly expect comparison with recent tabular deep learning, not only boosting.

ID	Baseline	Notes
B7	TabPFN (pretrained tabular foundation model)	Has limits on training rows and features: check the current version's limits, then use the B5 top features and an ensemble over stratified subsamples
B8	FT-Transformer	Transformer for tabular data trained from scratch; same features as B5
B9	CatBoost	Strong with high-cardinality categoricals such as email domain and DeviceInfo

- ☐ Give every baseline the **same tuning budget** (same number of Optuna trials, same validation period) and report it; unequal tuning is a common reviewer objection.
- ☐ Record training time and inference time per 1,000 transactions for every baseline.

Tasks

- ☐ Implement B1–B6 as harness configs (EXP-001 to EXP-006).

- ☐ V-column reduction: group V columns by identical missing pattern, keep one representative per highly correlated group ($|r| > 0.75$) inside each group.
- ☐ Tune B5 with Optuna on the validation period only (≈ 50 trials); fix the best parameters in the config.
- ☐ Imbalance: compare no weighting vs `scale_pos_weight` ; keep whichever wins on validation PR-AUC. Do not use SMOTE on the time-ordered data without a separate justified experiment.
- ☐ Produce Table 1 (baselines, mean $\pm$ std, 95% CI) automatically into `research/tables/`.
- ☐ Re-evaluate the FYP-1 model under the new temporal split; record how much its old numbers drop (useful defense material).

Deliverables

- Baseline table, tuned B5 model, B5 SHAP summary plot.

Exit gate

- B5 beats B3 with non-overlapping CIs. If it does not, recheck the uid and aggregation features before moving on.

Phase 3: Behavioral and velocity view

Goal: features that describe how unusual a transaction is for its own customer, all computed point-in-time.

Group	Features	Notes
Amount deviation	z-score of amount vs the uid's past amounts; ratio to past median; amount percentile within uid history	Use median/MAD as a robust alternative to mean/std
Velocity	transaction count and amount sum in the last 1 h, 24 h, 7 d, 30 d per uid	Windows on TransactionDT
Recency	seconds since the uid's previous transaction; mean gap of the uid	Short gaps after long silence are a known pattern
Escalation	slope of amount over the uid's last 5 transactions	Captures the "\$20, \$50, \$800" pattern

Group	Features	Notes
Novelty	first time this uid uses this email domain / device / addr2 / ProductCD	Binary flags + count of new attributes
Entity velocity	distinct uids per card, email, device in the last 7 d	Bridges to the graph view
History strength	number of past transactions of the uid, days since first seen	Feeds the fusion gate as context; 0 = cold start
Time pattern	hour-of-day from TransactionDT and its deviation from the uid's usual hour	Relative hour only; real local time is unknown (state as limitation)

The customer deviation score, for reference:

$$z_{\text{amt}} = \frac{a_t - \operatorname{median}(a_{\{<t\}})}{\operatorname{MAD}(a_{\{<t\}})} \cdot 1.4826 \cdot \epsilon$$

Tasks

- ☐ Implement each group in `features/behavioral.py` with unit tests on a hand-made 10-row example.
- ☐ Run both leakage tests on the new features.
- ☐ EXP: B5 + behavioral; report the gain with paired bootstrap.
- ☐ Report gains separately for uids with ≥ 5 past transactions and for cold-start uids.
- ☐ Keep the per-user Isolation Forest idea from FYP-1 for Phase 6, not here.

Deliverables

- Behavioral feature module, gain table, top-20 feature importance plot.

Exit gate

- Behavioral features improve PR-AUC over B5, or you can explain with evidence why they do not.

Phase 4: Dynamic graph view and leakage study (Paper 1)

Goal: a graph that only ever contains the past, graph features and a GNN built on it, and a measured answer to how much leakage inflates graph results.

4.1 Graph schema

Node type	Built from	Present for
Transaction	TransactionID	All rows
Customer	uid	All rows
Card	card1-card6 combined	All rows
Email	P_emaildomain, R_emaildomain	Rows where present
Address	addr1 + addr2	Rows where present
Device	DeviceInfo + DeviceType (+ selected id_ columns)	~24% of rows

Edges link each transaction to its entities and carry the transaction's timestamp.

4.2 Graph features (point-in-time)

- ☐ Degree of each entity at time t (how many past transactions and uids touched it).
- ☐ Shared-entity counts: other uids sharing this card, email, address or device.
- ☐ Neighbor known-fraud rate, respecting the label delay L .
- ☐ Two-hop known-fraud count (transaction $\rightarrow$ entity $\rightarrow$ other transaction).
- ☐ Connected component size and number of distinct uids in it (rolling 30-day window).
- ☐ Community ID and community known-fraud rate (Louvain on the 30-day window).

4.3 GNN

- ☐ Build PyTorch Geometric `HeteroData`; train heterogeneous GraphSAGE with `NeighborLoader` (2 layers, fanout 10/5 to start).
- ☐ Temporal sampling: when scoring a transaction at time t , sample only edges with time $< t$ and labels known by $t - L$.
- ☐ Output: graph-view fraud probability per transaction; missing (masked) when the transaction has no entity edges beyond its own uid and card.
- ☐ Compare: B5 + graph features (XGBoost) vs GNN alone vs both.

4.4 Leakage study (Paper 1 core)

Setting	Edges used	Labels used	Expected
A: Static full graph	All edges, past and future	All labels	Inflated
B: No future labels	All edges	Only labels known by $t - L$	Partly inflated
C: Strictly temporal	Only edges before t	Only labels known by $t - L$	Honest

- ☐ Run A, B and C with the same model, features and seeds; report the PR-AUC gap A – C and B – C with CIs.
- ☐ Repeat on the Elliptic dataset to show the effect is not specific to IEEE-CIS.
- ☐ From the literature spreadsheet, count how many graph fraud papers use a setup like A or B.

4.5 Fraud-ring module

- ☐ Flag components with ≥ 3 uids sharing a device, card or address within 30 days.
- ☐ Ring score = share of known fraud + density + growth speed.
- ☐ Evaluate: what share of later fraud falls inside rings flagged earlier.

4.6 Continuous-time dynamic GNN (added)

Snapshot GNNs rebuild the graph in steps; a continuous-time model keeps a memory per node and updates it with every transaction, which matches streaming fraud and the leakage theme of Paper 1.

- ☐ Implement a TGN-style model (node memory + temporal attention) with PyTorch Geometric's temporal modules (`TemporalData` , `TGNMemory`) on transaction $\rightarrow$ entity events.
- ☐ Strict ordering: memory is updated only after a transaction is scored, so a transaction never sees its own event.
- ☐ Labels enter memory only after the delay L .
- ☐ Compare under setting C: graph features + XGBoost vs snapshot GraphSAGE vs TGN-style model (E18).

- ☐ Report results by entity-link density (no shared entities, 1-2, 3+), since dynamic memory should help most where links are dense.
- ☐ Measure scoring latency per event; it decides whether TGN or GraphSAGE becomes the live graph view.

4.7 Elliptic replication

- ☐ Run settings A/B/C and the TGN-style model on the Elliptic Bitcoin graph (it has time steps), so Paper 1's leakage finding is shown on two datasets.

Deliverables

- Graph builder, graph features, GNN, A/B/C table, ring detector, Paper 1 draft results.

Exit gate

- Setting C runs end to end and its numbers are reproducible; the leakage gap is reported with CIs.

Phase 5: Temporal sequence view

Goal: a model that reads each customer's recent transaction sequence, with four architectures compared fairly.

Tasks

- ☐ Build sequences: for each transaction, the uid's previous N transactions (N = 10 and 20), each step = amount (log), time gap to next, ProductCD, novelty flags, a small set of C/D features.
- ☐ Left-pad short histories and pass a mask; uids with no history get **no** temporal score (masked view), not a fake one.
- ☐ Train four models with the same inputs and tuning budget: GRU, LSTM, TCN, small Transformer (2 layers, 4 heads).
- ☐ Loss: weighted binary cross-entropy; early stopping on validation PR-AUC.
- ☐ Compare the four on PR-AUC, latency per transaction and parameter count; keep the best as the temporal view.
- ☐ Report results by history length (1-4, 5-19, 20+ past transactions).

Deliverables

- Sequence builder, four trained models, comparison table, chosen temporal view.

Exit gate

- The chosen model adds measurable value on top of B5 + behavioral for uids with 5+ past transactions.

Phase 6: Anomaly view

Goal: a label-free score that can flag patterns the supervised models have never seen.

Tasks

- ☐ Global Isolation Forest trained on legitimate training transactions only.
- ☐ Autoencoder (3-layer MLP) trained on legitimate transactions; anomaly score = reconstruction error.
- ☐ Per-customer Isolation Forest from FYP-1, used only for uids with ≥ 20 past transactions (masked otherwise).
- ☐ Normalize all anomaly scores to $[0, 1]$ by rank on the training period.
- ☐ **Emerging-fraud experiment:** remove all fraud of one ProductCD (and, separately, one card4 type) from training, then measure how well each view detects that held-out fraud at test time. Repeat for each held-out group.

Deliverables

- Anomaly models, emerging-fraud table (view $\times$ held-out group).

Exit gate

- The anomaly view detects held-out fraud better than random for at least half the groups; if not, report it and keep the view as a fusion input only.

Phase 7: Missing-view-aware adaptive fusion (main contribution)

Goal: a fusion network that decides, per transaction, how much to trust each view given which views exist, how confident they are and the context, and that beats stacking.

7.1 Inputs to the gate

Input	Definition
View scores p_v	Calibrated probability from each of the 5 views (logit form)
Availability mask m_v	1 if the view produced a score, 0 if missing (no identity, no history, no graph edges)
View confidence	Distance from 0.5, or variance across the view's 5 seeds
Disagreement	Standard deviation of available view scores
Context	log amount, history length, has_identity, ProductCD, hour

7.2 The model

A small MLP (2 hidden layers of 32) outputs one weight per view. Missing views get weight 0 through a masked softmax, and the fused score is a weighted sum in logit space:

$$w_v = \frac{m_v \cdot e^{g_v(x)}}{\sum_u m_u \cdot e^{g_u(x)}}, \quad \hat{p} = \frac{\sum_v w_v \cdot \operatorname{logit}(p_v) + b(x)}{\sum_v w_v}$$

Here $g_v(x)$ is the gate's raw score for view v and $b(x)$ a learned context bias.

7.3 Training rules (avoid stacking leakage)

- ☐ Train each view on the training period; produce its predictions on the validation period (out-of-sample).
- ☐ Train the gate only on those validation-period predictions; evaluate on the test period.
- ☐ For the final model, use rolling out-of-time predictions so the gate sees enough data (document the scheme).
- ☐ **View dropout:** during gate training, hide each available view with probability 0.2–0.3 so the gate learns to cope with missing evidence.
- ☐ Loss: weighted BCE; variant with cost weights (missed fraud weighted by amount).

7.4 Fusion baselines (must all be beaten or honestly reported)

ID	Fusion method
F1	Simple average of available views
F2	Fixed weights tuned on validation
F3	Logistic-regression stacking (missing views filled with a constant)
F4	LightGBM stacking with missing values left as NaN
F5	Gate without the availability mask
F6	Gate with mask but without view dropout
MVAF	Full method

7.5 Analysis

- ☐ Results overall and by subgroup: has_identity yes/no, cold-start vs established uid, graph edges yes/no.
- ☐ Robustness test: at test time, artificially remove each view for all transactions; plot how much each fusion method drops.
- ☐ Plot the average gate weight per view by context (e.g. graph weight rises when shared devices exist). This is also an explanation of the model.

Deliverables

- MVAF code, fusion comparison table, subgroup table, robustness plot, gate-weight plots.

Exit gate

- MVAF vs F3/F4 compared with paired bootstrap on all subgroups. If MVAF does not win, Paper 2 becomes an honest analysis of fusion under missing views; do not tune on test to force a win.

Phase 8: Uncertainty, calibration and budget-aware routing

Goal: every score comes with a trustworthy confidence, and analysts' limited time goes to the cases where it saves the most money.

8.1 Calibration

- ☐ Fit isotonic and Platt calibration on validation; keep the one with lower ECE.
- ☐ Report Brier score, ECE (15 bins) and a reliability diagram, before and after calibration.

8.2 Conformal prediction

- ☐ Split conformal on a held-out calibration slice of validation; nonconformity = $1 - p(\text{true class})$.
- ☐ Class-conditional (Mondrian) conformal, because fraud is rare and needs its own coverage guarantee.
- ☐ Output a prediction set per transaction: {legit}, {fraud} or {legit, fraud}. The last means "uncertain" and goes to review.
- ☐ Adaptive conformal inference (online update of the error level) for the streaming setting; check coverage month by month.
- ☐ Report: empirical coverage vs target (90%, 95%), share of uncertain sets.

8.3 Cost-sensitive decision engine

Expected loss ($p \times \text{amount}$) and set	Action
Low risk, confident {legit}	Allow
Low-medium risk	Allow + monitor
Medium risk or uncertain set	Step-up verification (simulated OTP)
High risk or high disagreement	Hold + analyst review
Very high risk, confident {fraud}	Block + open case

Thresholds are chosen on validation by minimizing total cost (cost model in Phase 13), never set by hand.

8.4 Budget-aware routing (Pillar 4)

- ☐ Simulate analyst capacity K reviews per day ($K = 50, 100, 200, 500$).
- ☐ Policies: (R1) top-K by risk, (R2) top-K by expected loss, (R3) R2 + uncertainty bonus, (R4) R3 + disagreement bonus.
- ☐ Metrics: fraud value caught, fraud count caught, recall@K, money saved per review.

Deliverables

- Calibration report, conformal module, decision engine, routing comparison table and curve.

Exit gate

- Conformal coverage within 2 points of target on test; routing comparison done for all K.

Phase 9: Explainability and faithfulness

Goal: every alert explains itself at two levels (which views drove it, which features inside them), and the explanations are tested, not just displayed.

9.1 Explanation layers

Layer	Method	Shown to analyst as
View level	Gate weight $\times$ view score contribution	"Graph evidence 45%, behavior 30%, tabular 25%"
Feature level	SHAP (TreeExplainer) for tabular and behavioral views	Top 5 reason codes with signed points
Relational evidence	Shared entities with known fraud, ring membership	"Shares a device with 3 confirmed fraud accounts"
Temporal evidence	Gradient $\times$ input or attention over the sequence	"Amount jumped 25 $\times$ after 3 small transactions in 8 min"
Counterfactual	DiCE with immutable features locked (card, addr, history)	"From a known device and under \$120, risk falls to 31"

9.2 Reason codes

- ☐ Map each engineered feature to a plain-language template ("Amount is {z:.1f}× above this customer's usual").
- ☐ For masked raw features, use honest wording ("Unusual value in masked counter C13") rather than invented meanings.
- ☐ Case narrative: a template that combines decision, top reasons, graph evidence and counterfactual into one paragraph. Optional extension: a local LLM restricted to the same facts, tested for statements not supported by the evidence.

9.3 Faithfulness and stability (Pillar 5)

- ☐ Deletion test: replace the top-k features ($k = 1 \dots 10$) with training medians; plot risk drop; compare against random-k and against permutation importance.
- ☐ Insertion test: start from medians and add features back in SHAP order.
- ☐ Stability: explanations of near-identical transactions (small amount change) should share their top-3 reasons; report overlap (Jaccard).
- ☐ Seed consistency: top-5 reasons across the 5 seeds of the same model.
- ☐ Counterfactual quality: validity (does the risk really fall), proximity, number of features changed, and whether every changed feature is actionable.

Deliverables

- Explanation API, reason-code library, case narratives, faithfulness and stability table.

Exit gate

- SHAP deletion curves beat random deletion with CIs; counterfactual validity above 90% on a 500-alert sample.

Phase 10: Drift under delayed labels and continuous learning (Paper 3)

Goal: detect when fraud changes even though labels arrive late, and update the model only when a new one is proven better.

10.1 Drift signals

Signal	Needs labels?	Method
Feature drift	No	PSI per feature (alert > 0.2), KS test with Holm correction
Score drift	No	PSI on the fused score distribution
Disagreement drift	No	Rise in mean view disagreement and in share of uncertain conformal sets
Performance drift	Yes (delayed)	ADWIN and Page-Hinkley on the error stream of matured labels

10.2 Experiments

- ☐ Real drift: run the rolling monthly split; plot PR-AUC per month for a frozen model.
- ☐ Injected drift with a known start date, so detection delay can be measured: (a) shift fraud amounts in one ProductCD, (b) move fraud to a new email domain, (c) make fraud mimic legit velocity.
- ☐ Label delays $L = 0, 7, 30, 60$ days.
- ☐ For each detector: detection delay in days, false alarms per 6 months, missed drifts.

10.3 Retraining policies

Policy	Rule
P0	Never retrain
P1	Retrain every month
P2	Retrain when a label-free detector fires
P3	Retrain when a label-based detector fires

- ☐ Champion-challenger gate: a retrained model replaces the current one only if it beats it on the most recent matured-label window by a set margin (PR-AUC and cost).
- ☐ Compare policies on PR-AUC over time, total cost, and number of retrains.

10.4 Feedback and active learning

- ☐ Analyst labels from the dashboard (fraud / legit / unsure) are stored with time and analyst ID; "unsure" never enters training.
- ☐ Active learning simulation: each day reveal labels only for K selected transactions. Strategies: random, uncertainty, disagreement (query by committee), expected loss.
- ☐ Metric: PR-AUC after 3 months vs number of labels used; label saving at equal recall.

Deliverables

- Drift monitor, label-delay simulator, injected-drift results, policy comparison, active-learning curves, Paper 3 draft results.

Exit gate

- All detectors evaluated under all delays on the three injected drifts; policy comparison complete.

Phase 11: Real-time platform

Goal: the research models running as a live system that replays transactions, scores them in real time and stores every decision, with measured latency.

11.1 Streaming

- ☐ Kafka in Docker (single node, KRaft). Topics: `tx.raw`, `tx.scored`, `alerts`, `feedback`.
- ☐ Replay producer: reads the test period in TransactionDT order and publishes at a chosen speed-up ($\times 1$, $\times 100$, $\times 1000$).
- ☐ Scorer consumer: reads `tx.raw`, updates online features, calls the models, writes to `tx.scored` and PostgreSQL, sends `alerts`.

11.2 Online features

- ☐ Redis holds rolling counters and last-seen values per uid and per entity (card, email, device, address).
- ☐ **Parity test:** for 10,000 replayed transactions, online feature values must equal the offline pipeline's values (tolerance $1e-6$). Mismatch here means research results and the live system disagree.

11.3 API (FastAPI)

Endpoint	Purpose
POST /score	Score one transaction; returns risk, probability, confidence set, view scores, decision
GET /explain/{id}	Reasons, view weights, graph evidence, counterfactual, narrative
GET /graph/{entity}	Neighborhood for the fraud-network page
POST /feedback	Analyst label
GET /drift	Latest drift report
GET /models	Model registry: versions, metrics, champion
GET /health, /metrics	Health and Prometheus-style metrics

11.4 Database, security, packaging

- ☐ PostgreSQL tables: transactions, scores, view_scores, decisions, alerts, cases, feedback, model_registry, drift_reports, audit_log.
- ☐ JWT login with roles (analyst, admin); hash passwords; encrypt sensitive fields with the FYP-1 `crypto_utils`; audit every analyst action.
- ☐ Keep FYP-1 features working: email alerts, PDF reports.
- ☐ `docker compose up` starts Kafka, Redis, PostgreSQL, API, scorer, MLflow and dashboard.
- ☐ GitHub Actions: tests, leakage tests, parity test on a small sample, lint.

11.5 Performance benchmark

- ☐ Locust load test at increasing rates (e.g. 50, 100, 250, 500, 1,000 TPS) until p99 latency or error rate breaks.
- ☐ Report p50/p95/p99 end-to-end latency per stage and the maximum stable TPS on your hardware (state the CPU/RAM).

Exit gate

- Parity test passes; benchmark table produced; the full system starts with one command on a teammate's laptop.

Phase 12: Dashboard

Goal: a security-operations-style dashboard that makes every research component visible and usable in the defense demo.

Page	Shows	Research link
Overview	Transactions today, alerts, fraud value blocked, estimated loss, analyst queue size	Cost evaluation
Live monitor	Streaming table: risk, confidence set, decision, top reason; filters	Decision engine
Case investigation	Risk gauge, view scores and gate weights, SHAP reasons, customer timeline, mini graph, counterfactual, narrative, feedback buttons	Pillars 1, 5
Fraud network	Interactive Cytoscape graph; click a node to expand; rings highlighted	Pillar 2
Analyst queue	Today's K cases chosen by the routing policy; switch policies to compare	Pillar 4
Drift monitor	PSI per feature, score drift, disagreement trend, detector alarms, retraining history	Pillar 3
Model lab	Pick any experiment run; compare metrics, PR curves, calibration, ablation table	All RQs
Scenario simulator	Sliders for amount, device novelty, velocity, shared entities; live risk and reasons	Explanations, demo
Reports and settings	PDF reports, user roles, thresholds (admin only)	FYP-1 features

Tasks

- ☐ React + Vite app; Recharts for charts, Cytoscape.js for graphs, TanStack Table for tables.
- ☐ Live updates through WebSocket or server-sent events from the API.
- ☐ Responsive layout down to tablet width; dark and light themes.
- ☐ Usability check: 3–5 people (classmates acting as analysts) complete 5 investigation tasks; record time and errors. This becomes a small user-study section.

Exit gate

- Every page works on the replayed stream; a full demo script runs start to finish in 10 minutes.

Phase 13: Full experiment suite

Goal: one final, clean run of every experiment from configs, producing every table and figure for the papers and thesis.

ID	Experiment	Answers	Paper
E1	Baselines B1–B6	Reference	2
E2	FYP-1 model under temporal split	Shows why rigor matters	Thesis
E3	B5 + behavioral	RQ1	2
E4	+ temporal view	RQ1	2
E5	+ graph features	RQ1	1, 2
E6	Graph leakage settings A/B/C	RQ3	1
E7	GNN vs graph features	RQ1, RQ3	1
E8	+ anomaly view	RQ1	2
E9	Emerging (held-out) fraud by view	RQ1	2
E10	Fusion F1–F6 vs MVAF	RQ2	2
E11	Missing-view robustness and subgroups	RQ2	2

ID	Experiment	Answers	Paper
E12	Calibration, conformal coverage, routing policies	RQ5	2
E13	Drift detectors × label delays × injected drifts	RQ4	3
E14	Retraining policies and active learning	RQ4	3
E15	Explanation faithfulness, stability, counterfactual quality	RQ6	2
E16	Latency and throughput benchmark	System	2, Thesis
E17	Usability study	System	Thesis

Added experiments:

ID	Experiment	Answers	Paper
E18	Graph features vs GraphSAGE vs TGN-style model, strict temporal setting	RQ3, RQ7	1
E19	Modern tabular baselines B7–B9 under equal tuning budget	Reference	2
E20	LLM copilot grounding: grounded vs ungrounded answers	RQ8	4 (new)
E21	Adversarial evasion and adversarial retraining	RQ9	2 or 5
E22	Federated vs centralized vs local training	RQ10	5 (new)
E23	Subgroup performance gaps (ProductCD, device type, email domain, cold start)	Fairness check	2
E24	Compute and carbon cost per model (training hours, hardware)	Reporting	2

Metrics

- Detection: PR-AUC (primary), ROC-AUC, recall at 1% FPR, precision@K, F1 at validation threshold.
- Calibration: Brier score, ECE, conformal coverage.

- Business: total cost, fraud value caught, money saved per analyst review.
- System: p50/p95/p99 latency, maximum stable TPS.

Cost model

$$\text{Cost} = \sum_{i} \{\text{missed fraud}\} \text{amount}_i + C_{\text{FP}} \cdot \#\text{FP} + C_{\text{rev}} \cdot \#\text{reviews}$$

Use $C_{\text{FP}} = \$10$ and $C_{\text{rev}} = \$5$ as stated assumptions, plus a sensitivity analysis over \$2-\$50.

Statistics rules

- ☐ Five seeds per configuration; report mean $\pm$ std.
- ☐ 95% bootstrap CIs (1,000 resamples) on the test period.
- ☐ Paired bootstrap test for every "A beats B" claim; Holm correction when several comparisons share a table.
- ☐ Effect sizes reported, not only p-values.

Main ablation table (fill only from the harness)

Model	Behavioral	Temporal	Graph	Anomaly	Fusion	PR-AUC	Recall@1%FPR	Cost
B5 strong tabular	no	no	no	no	none			
+ behavioral	yes	no	no	no	none			
+ temporal	yes	yes	no	no	stacking			
+ graph	yes	yes	yes	no	stacking			
+ anomaly	yes	yes	yes	yes	stacking			
MVAF (full)	yes	yes	yes	yes	MVAF			

Exit gate

- `make paper-results` regenerates every table and figure from scratch without manual steps.

Phase 14: Papers, thesis and defense

Goal: three papers submitted in the order their results become ready, a thesis built from the same material, and a defense with no question left unprepared.

Papers

Paper	Working title	Ready after	Core content
1	Temporal leakage in graph-based fraud detection evaluation	Phase 4	A/B/C settings on IEEE-CIS and Elliptic, literature audit of graph setups, recommended protocol
2	Missing-view-aware adaptive fusion for explainable, uncertainty-aware fraud detection	Phases 7–9	MVAF, fusion baselines, subgroups, routing, faithfulness, full ablation
3	Detecting fraud drift under delayed labels	Phase 10	Detectors × delays × injected drifts, retraining policies, active learning

Each paper follows: abstract, introduction (problem, gap, contributions as 3–4 bullets), related work (from the spreadsheet), method, experimental setup (data, split, leakage rules, metrics, statistics), results, ablation, discussion, threats to validity, conclusion.

Choosing venues

- ☐ Shortlist venues with your supervisor; check that each is indexed in Scopus or Web of Science and is not on predatory-journal warning lists.
- ☐ Conferences for Papers 1 and 3 (faster), a journal for Paper 2.
- ☐ Read each venue's page limit and template before writing, not after.
- ☐ Release code and configs publicly on GitHub at submission, with a README for reproducing every table.

Thesis chapters

1. Introduction · 2. Literature review · 3. Data and leakage-safe methodology · 4. Evidence views · 5. MVAF fusion · 6. Trust layer (uncertainty, decisions, explanations) · 7. Drift and continuous learning · 8. System design and implementation · 9. Experiments and

results · 10. Conclusion, limitations, future work · Appendices (configs, API, user manual).

Defense preparation

- ☐ 10-minute live demo script + a recorded backup video in case the live system fails.
- ☐ One slide per pillar: question, method, result, limitation.
- ☐ Prepare answers, with numbers, to likely questions:
 - Why not random splitting? How do you know there is no leakage?
 - How was the customer ID built, and how reliable is it?
 - Why does MVAF beat (or not beat) plain stacking?
 - What happens when the device data is missing?
 - Are SHAP explanations faithful? How did you test it?
 - How are thresholds chosen? What are the cost assumptions?
 - How do you detect drift if labels arrive late?
 - Can this run in a real bank? What is missing (real data, compliance, scale)?
 - What is your own contribution versus existing libraries?
 - What would you do with six more months?
- ☐ Mock defense with the supervisor or seniors at least twice.

Exit gate

- Paper 1 submitted; Papers 2 and 3 drafted; thesis approved; demo rehearsed.

Phase 15: LLM investigation copilot (core extension)

Goal: an analyst assistant that answers questions about a case using only evidence fetched from Vaultic, with a measured rate of unsupported statements.

15.1 Design

Part	Choice
Model	A free open-weight model run locally through Ollama (e.g. a 7-8B instruct model); no paid API required

Part	Choice
Tools the LLM can call	<code>get_case(id)</code> , <code>get_reasons(id)</code> , <code>get_view_scores(id)</code> , <code>get_graph(entity, hops)</code> , <code>get_customer_history(uid, n)</code> , <code>get_counterfactual(id)</code> , <code>get_similar_cases(id, k)</code>
Rules	Answer only from tool results; every sentence cites the tool result it used; say "not available" otherwise; never change decisions or labels
Output	Answer, cited evidence IDs, and a confidence note

15.2 Features in the dashboard

- ☐ Chat panel on the case page: "Why was this flagged?", "Which accounts share this device?", "Has this customer done this before?"
- ☐ One-click case summary for the case file and PDF report.
- ☐ Suggested next checks for the analyst (from rules, not invented by the model).

15.3 Evaluation (E20, possible Paper 4)

- ☐ Build a test set of 200 cases × 5 question types with reference answers written from system evidence.
- ☐ Compare: (a) LLM with tools, (b) LLM given the raw case text only, (c) template narrative from Phase 9.
- ☐ Metrics: answer accuracy, unsupported-claim rate (each claim checked against evidence), citation correctness, latency.
- ☐ Two team members label a 50-case sample independently; report agreement (Cohen's kappa).
- ☐ Safety tests: prompts that try to make it change a decision, reveal other customers' data, or ignore its rules.

Exit gate

- Grounded mode measured against both alternatives on the full test set; safety tests documented.

Phase 16: Adversarial robustness (stretch)

Goal: measure how much detection drops when fraudsters adapt within realistic limits, and whether retraining on attacks fixes it.

- ☐ Define a threat model: what an attacker **can** change (amount, timing between transactions, which device or email they use) and what they **cannot** (card history, past graph links, masked bank-side fields).
- ☐ Attack 1: greedy search over allowed changes to lower the risk score below the block threshold, with a cost limit (e.g. amount may change $\pm 20\%$).
- ☐ Attack 2: a small generator network trained to produce evasive fraud within the same limits.
- ☐ Measure recall under attack per view and for MVAf; count how many changes each successful evasion needed.
- ☐ Adversarial retraining: add successful evasions (labelled fraud) to training; re-measure clean and attacked recall.
- ☐ Exit gate: attack and defense results for the tabular model and MVAf (E21).

Phase 17: Federated multi-bank training (stretch)

Goal: show the framework can be trained across institutions that cannot share transactions.

- ☐ Split IEEE-CIS into 3–5 simulated banks by card4/card6 or email-domain groups, giving each bank a different data mix (non-identical distributions).
- ☐ Use the Flower framework (free) for federated training of the fusion gate and a neural tabular view; tree models stay local.
- ☐ Compare: each bank alone, federated (FedAvg and FedProx), centralized (upper bound).
- ☐ Optional: add differential privacy to updates and report the privacy–accuracy trade-off.
- ☐ Exit gate: three-way comparison with CIs (E22).

Publication readiness checklist

Most rejections come from missing rigor, not weak models; check every item below before any paper is submitted.

Novelty protection

- ☐ **Novelty check at three points:** before Phase 4, before Phase 7 and before each submission, search Google Scholar, arXiv and Semantic Scholar for the last 12 months on each pillar. If someone published the same idea, adjust the angle and cite them.
- ☐ Write each paper's contribution list before running its final experiments, so experiments test claims instead of claims being fitted to results.
- ☐ Keep dated lab notes and commits; they prove when your work was done.

Rigor

- ☐ Equal tuning budget for every compared method, reported in the paper.
- ☐ All thresholds and hyperparameters chosen on validation; the test period is touched only for final runs.
- ☐ Five seeds, bootstrap CIs, paired tests, Holm correction (Phase 13).
- ☐ Report negative results and failed ideas in the discussion or appendix.
- ☐ Report compute: hardware, training hours per model (E24).

Responsible AI

- ☐ Subgroup analysis (E23): performance gaps across ProductCD, device type, email-domain groups and cold-start customers; IEEE-CIS has no demographic data, so state that real fairness auditing needs it.
- ☐ Ethics statement: public anonymized data, no personal data collected, risk that fraud models wrongly block legitimate customers, and how human review limits that.
- ☐ Model card (what the model is for, data, metrics, limits) and a data datasheet in the repo.

Reproducibility package

- ☐ Public GitHub release with a DOI (via Zenodo) at submission time.
- ☐ `make paper-results` reproduces every table; environment pinned with Docker.
- ☐ README section per paper: commands, expected runtime, expected numbers.

Writing quality

- ☐ Each paper reviewed by the supervisor and at least one outsider before submission.

- ☐ Similarity check (Turnitin or the university's tool) under the allowed limit; every reused sentence quoted and cited.
- ☐ References managed in Zotero; every claim about other work checked against the original paper.

Possible papers (updated)

Paper	Topic	Source phases
1	Temporal leakage in graph fraud detection evaluation (+ TGN comparison)	4
2	Missing-view-aware adaptive fusion (main)	2–9, 13
3	Drift detection under delayed fraud labels	10
4	Grounded LLM copilot for fraud investigation	15
5	Federated or adversarially robust fraud detection (if stretch phases are done)	16–17

Team division and milestones

Each person owns whole phases end to end and leads one paper; everyone reviews everyone's pull requests.

Person	Owns	Leads	Supports
Aziz	Phase 1 data foundation, Phase 2 baselines, Phase 3 behavioral, Phase 7 MVAF fusion, Phase 8 uncertainty and routing	Paper 2, thesis integration	Phase 9 research side
Hamza	Phase 4 graph and leakage study, Phase 5 temporal, Phase 6 anomaly, Phase 10 drift and active learning	Papers 1 and 3	Literature review
Roshan	Phase 9 explanation integration, Phase 11 platform, Phase 12 dashboard, usability study	System chapter, demo	Online feature parity
All	Phase 0, Phase 13, Phase 14		

If the split feels uneven once work starts, move Phase 6 or Phase 10 to whoever finishes first. Agree on this at a weekly meeting, not informally.

Working rhythm

- Weekly 1-hour meeting: each person shows one result, one blocker, next week's goal.
- Every result shown at the meeting must come from the harness with its experiment ID.
- Update this document's checkboxes as tasks finish.

Milestone checklist

- ☐ M0: Repo, CI, literature spreadsheet, supervisor approval (Phase 0)
- ☐ M1: Leakage tests pass, uid chosen, harness reproducible (Phase 1)
- ☐ M2: Baseline table with CIs (Phase 2)
- ☐ M3: Behavioral view measured (Phase 3)
- ☐ M4: Graph A/B/C results; Paper 1 drafted (Phase 4)
- ☐ M5: Temporal and anomaly views measured (Phases 5–6)
- ☐ M6: MVAF vs all fusion baselines (Phase 7)
- ☐ M7: Conformal coverage and routing results (Phase 8)
- ☐ M8: Faithfulness results and explanation API (Phase 9)
- ☐ M9: Drift and retraining results; Paper 3 drafted (Phase 10)

- ☐ M10: Live system with parity test and benchmark (Phase 11)
- ☐ M11: Dashboard complete, usability study done (Phase 12)
- ☐ M12: All tables regenerate from one command (Phase 13)
- ☐ M13: Papers submitted, thesis approved, defense rehearsed (Phase 14)

Added work and milestones

Added work	Owner
Modern baselines B7-B9 (Phase 2)	Aziz
TGN-style model and Elliptic replication (Phase 4.6-4.7)	Hamza
LLM copilot (Phase 15)	Roshan (tools and UI), Aziz (evaluation)
Adversarial robustness (Phase 16)	Aziz
Federated training (Phase 17)	Hamza
Publication readiness checklist	All, before each submission

- ☐ M14: TGN-style model compared under strict temporal setting (E18)
- ☐ M15: Copilot evaluated on 200 cases with safety tests (E20)
- ☐ M16: Stretch phases 16-17 done, or formally dropped with a written reason

Risks, fallbacks and wording rules

The biggest risk is a weak foundation (customer ID or leakage), so it is checked first and tested automatically for the whole project.

Risk	Early sign	Fallback
Reconstructed uid is unreliable	Low fraud purity per uid, many one-transaction IDs	Use the best variant, report its quality as a limitation, add card-level features as backup
Leakage found late	Results suddenly drop when a test is added	Leakage tests in CI from Phase 1; re-run affected experiments from configs

Risk	Early sign	Fallback
GNN runs out of memory	Crashes or hours per epoch	Smaller fanout, 30-day subgraphs, or graph features + XGBoost only (still answers RQ3)
MVAF does not beat stacking	Paired bootstrap shows no difference	Report honestly; reframe Paper 2 as an analysis of fusion under missing views with the robustness results
Temporal model adds nothing	No gain over behavioral features	Keep the comparison as a finding; drop the view from the final system
Kafka/Docker problems on Windows	Containers fail to start	WSL2 + Docker Desktop; fall back to a Redis Streams queue with the same interfaces
Paper rejected	Reviews	Address every comment, submit to the next venue on the shortlist
A teammate is blocked or leaves	Missed milestones two weeks in a row	Reassign phases at the weekly meeting; the harness and docs make handover possible

Wording rules for papers, thesis and defense

Never write	Write instead
"Real bank data"	"Anonymized e-commerce transactions released by Vesta for the IEEE-CIS competition"
"99% accuracy"	PR-AUC and recall at a fixed FPR, with CIs
"PCI-DSS compliant"	"Security controls inspired by PCI-DSS principles"
"Handles 10,000 TPS"	The measured maximum stable TPS and p99 latency on stated hardware
"First ever / novel GNN approach"	The specific gap from your literature review and how this work addresses it

Never write	Write instead
"Detects all fraud"	Recall, false positive rate and cost at the chosen operating point
"Real-time" with no numbers	"Median end-to-end latency of X ms (p99 Y ms)"